



Procesos e Hilos

Procesos



Un **proceso** es un programa en ejecución junto con todos los recursos necesarios para que ese programa pueda ejecutarse. Por eso se le llama *entidad pesada*

Explicación

Un proceso no es solo “el código del programa”. Necesita:

- un **contador de programa (PC)** para saber qué instrucción ejecutar,
- un **conjunto de registros**,
- una **pila (stack)**,
- un **segmento de datos** (variables, memoria dinámica),
- y otros recursos del sistema operativo (archivos abiertos, permisos, etc.).

El programa sería algo estático. El proceso algo dinámico.

Cuando un programa empieza a ejecutarse, se crea al menos **un proceso** (o sea que todo programa tiene, al menos, un proceso en ejecución si no está bloqueado), y este proceso puede contener **uno o varios hilos** ejecutándose en paralelo o en concurrencia.

Hilos



Un **hilo** ("thread") es una *secuencia de control*: el camino independiente que sigue el programa dentro de un proceso. Es una *entidad ligera* que comparte casi todo con otros hilos del mismo proceso... excepto su propio estado de ejecución.

Cada hilo dentro de un proceso tiene:

- su **propio contador de programa**, porque está siguiendo su propio camino de ejecución;
- su **propia pila de llamadas**;
- sus **propios registros**.

Pero comparte con el resto de hilos de ese proceso:

- el **código**,
- el **heap (datos globales y objetos)**,
- los **ficheros abiertos**,
- y en general **todo el espacio de memoria del proceso**.

Esto es lo que hace a los hilos *ligeros* y a los procesos *pesados*.

La relación entre programa, proceso e hilo

- Un **programa** es estático. Es la receta.
- Un **proceso** es dinámico. Es la receta en ejecución.
- Un **hilo** es una de las manos que ejecutan la receta.

Un programa puede generar **uno o varios procesos**, por ejemplo ejecutándose varias veces simultáneamente.

Un proceso puede contener **uno o varios hilos**, que cooperan compartiendo datos pero ejecutándose de manera independiente.

Por estas relaciones, podemos tener distintos tipos de concurrencia en términos conceptuales.

¿Qué comparten los hilos? ¿Qué NO comparten?

✓ Comparten automáticamente:

- código del programa,
- variables globales,
- objetos del heap,
- recursos del proceso (ficheros abiertos, descriptores, sockets...).

En un sistema de memoria compartida **no se crean copias privadas de las variables**:

todos los hilos ven la **misma** versión de las variables del proceso. Por esto se producen los problemas.

✗ No comparten:

Cada hilo tiene su **propio estado de ejecución**, que incluye:

- contador del programa (PC),
- pila de ejecución (cada llamada de función va a su propia pila),
- registros internos del procesador.

Por esto varios hilos pueden estar ejecutando **diferentes partes del mismo método simultáneamente** sin confundirse.

Tipos de concurrencia que pueden aparecer

• Concurrencia entre procesos independientes.

Cada proceso tiene su propia memoria. Se comunican mediante pipes, sockets, ficheros...

• Concurrencia entre hilos de un mismo proceso.

Más eficiente, comparten memoria y comunicación inmediata, pero requiere mecanismos de sincronización (monitores, locks, semáforos...) para evitar condiciones de carrera.